



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next;
7  };
8
9  struct Node *head = NULL;
10
11 void insert_begin(int data) {
12     struct Node *n = (struct Node*)malloc(sizeof(struct Node));
13     n->data = data;
14     n->next = head;
15     head = n;
16 }
17
18 void insert_end(int data) {
19     struct Node *n = (struct Node*)malloc(sizeof(struct Node));
```

List: 20 -> 10 -> 30 -> 40 -> NULL
After delete 20: 10 -> 30 -> 40 -> NULL

=== Code Execution Successful ===

```
struct Node *temp = head;  
n->data = data;  
n->next = NULL;
```

```
if (head == NULL) {  
    head = n;  
    return;  
}
```

```
while (temp->next != NULL)  
    temp = temp->next;
```

```
temp->next = n;
```

```
}
```

```
void delete_node(int data) {  
    struct Node *temp = head, *prev = NULL;  
  
    if (head == NULL) return;
```

```
if (head->data == data) {  
    head = head->next;  
    free(temp);  
    return;  
}
```

```
while (temp != NULL && temp->data != data) {  
    prev = temp;  
    temp = temp->next;  
}
```

```
if (temp == NULL) return;
```

```
prev->next = temp->next;  
free(temp);
```

```
}
```

```
void traverse() {  
    struct Node *temp = head;
```

```
    }  
    printf("NULL\n");  
}
```

```
int main() {  
    insert_begin(10);  
    insert_begin(20);  
    insert_end(30);  
    insert_end(40);  
  
    printf("List: ");  
    traverse();  
  
    delete_node(20);  
    printf("After delete 20: ");  
    traverse();  
  
    return 0;  
}
```

Single Linked List

~~Linked List~~

Algorithm:

1. start
2. create a node structure with data and next
3. Initialize head = NULL

Insert at Beginning

1. Create a new node
2. Assign data to the node
3. Set newnode $\rightarrow$ next = head
4. Set head = newnode

Insert at End

1. create a new node
2. The list is empty, make head = new node
3. Else traverse till last node.
4. Set last $\rightarrow$ next = new node

Delete Node

1. If list is empty, print message
2. If the first node matches, update head.
3. Else search for data
4. Adjust links and free memory.

Traverse

1. start from head
2. Print each node's data
3. Move to next until NULL.

Exit